Randomized Quickselect vs Median-of-Medians

Ethan Fernandez · Advanced Algorithms · Programming Assignment 2 (Solo)

Repository: https://github.com/EJCFernandez/AdvAlgos_A2_Median_Selection

Video (unlisted YouTube): https://youtu.be/YML36Mb1mdU

1. The problem and the algorithms

1.1 The selection problem

Given an array A of n keys from a totally ordered set with potential repeated values and a rank k with $0 \le k \le n - 1$, the selection problem asks for the key that would sit at index k if A were sorted. The median, $k = \lfloor n/2 \rfloor$, is the case all my main experiments use. Sorting solves it in $O(n \log n)$ time but places every key when only one matters, and since any comparison-based algorithm needs at least $n - 1$ comparisons, linear time is the target.

Both algorithms share one skeleton. They keep a window [lo, hi] known to contain position k, choose a pivot from it and partition the window around that pivot. Only the part containing k is kept, and the loop repeats on the smaller window. Unlike quicksort, which must continue into both sides, selection continues into one.

The default partition I used is three-way which splits the window into keys less than, equal to and greater than the pivot, and stops if k lands in the equal block. The two-way Lomuto partition (Cormen et al., 2022, §7.1), which sends keys equal to the pivot left, is kept as a variant because of how badly it behaves on duplicates (Sections 2.3 and 4.6). Both algorithms work in place and use identical copies of the partition code, so they differ only in how the pivot is chosen. Additionally, both implementations began from the GeeksforGeeks versions (GeeksforGeeks, 2024, 2025), which I cross-checked against explanations on Stack Overflow (Edge, 2012; simon, 2012) before rewriting them as iterative, in-place functions that worked within the skeleton.

1.2 Randomized quickselect

The partition-and-narrow idea is due to Hoare (1961), and the version here picks the pivot uniformly at random from the window. A random pivot only needs to be good often enough as with probability ½ it falls in the middle half of the window, leaving at most ¾ of it. The window therefore shrinks geometrically on average, giving expected $\Theta(n)$ time, about 3.39n comparisons for the median with two-way partitioning (Knuth, 1972). Each pivot costs one random number, so the constant factor is small. Nothing is guaranteed on a single run, though, because a long sequence of pivots near the window's ends costs $\Theta(n^2)$, and while that is very unlikely, the running time remains a random variable even on one fixed input.

1.3 Median-of-medians (BFPRT)

Blum et al. (1973) showed that selection takes worst-case linear time without randomness, by spending work on a pivot that is guaranteed to be good. Each round splits the window into groups of g = 5 keys, sorts each group, collects the roughly n/5 group medians at the front of the window and calls the selection algorithm on those medians to find their median, x, which becomes the pivot.

At least half the group medians are at least x, and each of their groups contributes three keys at least x, so roughly 3n/10 keys are at least x so by symmetry, roughly 3n/10 are at most x. The kept side therefore holds at most about 7n/10 keys, so $T(n) \leq T(n/5) + T(7n/10) + O(n)$. Because $1/5 + 7/10 < 1$, this is O(n) on every input (Cormen et al., 2022, §9.3).

The guarantee has two costs. Every round pays for the group sorts and the recursive call, so the constant factor is several times quickselect's. And the 3n/10 argument only bounds the kept side if keys equal to the pivot are split off, which three-way partitioning does and Lomuto does not (Section 2.1). The group size g is a parameter: g = 3 weakens the bound to O(n log n), while g = 7 stays linear (Section 2.3).

1.4 How they differ, and the baseline

The two algorithms answer one question differently, where does a good pivot come from? Quickselect draws one almost for free and relies on probability whereas median-of-medians builds one at cost and relies on a proof. Quickselect has no bad inputs, only bad luck, so its risk lives in the tail of its running-time distribution. Median-of-medians doesn’t depend on luck but pays the full cost on every run.

As a reference point I also time std::nth_element from GCC's libstdc++, an introselect (Musser, 1997): a quickselect with a deterministic median-of-three pivot and a Hoare-style partition, falling back to O(n log n) heap-based selection if the recursion gets too deep (Sayle & Carlini, 2006). It is a tuned engineering baseline, not one of the two compared algorithms.

2. Theoretical comparison and predictions

This section was written before any timing runs and hasn't been edited since. While this section does involve predictions I do have a reasonable amount of knowledge in regard to how the theory works but how these ideas translate into the practical side are the main ideas.

2.1 Asymptotic comparison

Both algorithms (Hoare, 1961; Blum et al., 1973) run in expected linear time but differ in what they guarantee and in what that guarantee costs. Quickselect makes no promise on any single run whereas median-of-medians promises linear time on every run but pays for it in other ways.

Property	Randomized quickselect	Median-of-medians (g = 5)
Expected time	Θ(n)	Θ(n)
Worst-case time, three-way partition	Θ(n^2), with vanishing probability	Θ(n), guaranteed
Worst case, Lomuto, all-equal input	Θ(n^2) for every seed	Faster than any polynomial (2.3)
Comparisons for the median, random input	About 3.39n expected (Lomuto)	At most 30n (Lomuto) or 40n (three-way)
Extra space	O(1): iterative loop	O(log n) stack for the pivot recursion
Variation between runs on one input	Comparison count is a random variable	None: same input, same count

Quickselect. A uniformly random pivot lands in the middle half of the window with probability ½, which leaves at most ¾ of the window. The window therefore shrinks geometrically in expectation, giving O(n). With Lomuto (Cormen et al., 2022, Section 7.1), a window of m elements costs m - 1 comparisons and the classical expected total for rank k (Knuth, 1972) is:

$$C(n,k) \approx 2n + 2k\ln\left(\frac{n}{k}\right) + 2(n-k)\ln\left(\frac{n}{n-k}\right)$$

This gives about 2n for the minimum or maximum and 2(1 + ln 2)n ≈ 3.39n for the median. The worst case needs a long run of pivots near the window's ends. On distinct keys each bad pivot has probability at most about ½, so a run costing c times the expected amount should become exponentially unlikely as c grows.

Median-of-medians. At least half of the n/5 group medians are at least the pivot x and each of their groups contributes 3 such elements so at least about 3n/10 elements are at least x likewise at most x. The kept side therefore has at most 7n/10 + 6 elements (Blum et al., 1973; Cormen et al., 2022, Section 9.3), giving:

$$T(n) \leq T(n/5) + T(7n/10 + 6) + O(n)$$

Since 1/5 + 7/10 = 9/10 < 1, T(n) = O(n). With three-way partitioning this holds for any input with duplicates included and keys equal to x are removed, so the kept side contains only keys strictly below x (or strictly above) and at most 7n/10 + 6 of those exist.

The worst-case comparison bound follows from the same recurrence. Insertion sort on a group of 5 costs at most 10 comparisons (2 per element) and the partition costs 1 per element for Lomuto and at most 2 for three-way. So $T(n) \leq 3n + T(n/5) + T(7n/10)$ therefore giving $T(n) \leq 30n$ and with three-way, 40n. These are guarantees, not estimates. On random input the kept side is closer to n/2 than to 7n/10 so the typical cost should sit well below the bound.

2.2 Constant factors and memory

Median-of-medians does several passes over the data per round where quickselect does one, so its constant should be several times larger. Both are in place and allocate nothing which was a deliberate design choice (Section 1) so differences in time come from passes over memory, branches and comparisons, not from allocation.

- **Passes per round.** Quickselect makes one partition pass. Median-of-medians makes a group-sort pass, a pass gathering one median per group into a[lo..], a recursive selection on those n/5 medians and then the same partition pass.
- **Comparisons in group sorts.** Inserting the i-th element in insertion sort costs $(i + 1)/2 - 1/i$ comparisons on average. Per element, that is about 0.89 for g = 3, 1.54 for g = 5 and 2.13 for g = 7, on every level, on top of the partition.
- **A rough model for random input.** If the pivot lands near the true median, each level keeps about half, so $T(n) \approx cn + T(n/5) + T(n/2) = cn/0.3$. With c = 1.54 + 1 (Lomuto) this gives about 8.5n; with three-way at about 1.5 comparisons per element, about 10.1n.
- **Memory.** At $n = 10^7$ the array is 80 MB of 8-byte keys, far larger than any cache. Both partitions scan sequentially, which the prefetcher handles well, so I expect time at large n to track the number of element visits at least as much as the number of comparisons.
- **Branches.** On random data, each comparison against a pivot near the median is close to a coin flip for the branch predictor. Median-of-medians adds the branches of many tiny insertion sorts.
- **std::nth_element.** Assuming GCC's libstdc++ in WSL, this is introselect (Musser, 1997): a median-of-three pivot, a two-pointer Hoare-style partition, a heap-select fallback when the recursion gets too deep and insertion sort for small ranges (Sayle & Carlini, 2006). It differs from my quickselect in pivot rule, partition scheme and tuning, so it is a reference for what engineered code achieves, not a third algorithm.

2.3 Input features each algorithm should be sensitive to

Quickselect's comparison count depends only on the pivot draws and on k, not on the input order. Median-of-medians' count depends on the input order through its group sorts and on duplicates through the partition scheme.

Input feature	Randomized quickselect	Median-of-medians (g = 5)
Sorted or reverse-sorted order	Same comparison distribution as random input, since the pivot's rank is uniform either way time may still differ through branch prediction	Sorted groups cost 4 comparisons each (0.8 per element), reverse groups 10 (2 per element), random about 7.7 (1.54 per element)
All equal, three-way	Exactly one partition: 2n comparisons	2.8n + T(n/5), which gives 3.5n
All equal, Lomuto	One element removed per round: $(n^2 - k^2)/2$ comparisons, about $3n^2/8$ for the median	Faster than any polynomial (below)
Few distinct values, Lomuto	$\Theta(n^2)$ once the window holds only copies of the target value	Same blow-up on a smaller window
Position of k	About 2n at the ends, 3.39n at the median	Pivot choice ignores k
Group size g	Not applicable	g = 3 loses the linear guarantee; g = 7 keeps it with more sorting per level

All-equal input under Lomuto. Lomuto with ≤ sends every key equal to the pivot left, so the pivot ends at hi and each round removes one element. Selecting the median of a window of m elements takes about m/2 rounds and every round runs median-of-medians on its window, which selects the median of about j/5 medians with the same problem. So the cost S(m) satisfies roughly $S(m) \geq (m/2) \cdot S(m/10)$. Unrolling this over log m levels gives $S(m) = m^{\Theta(\log m)}$ so the exponent itself grows with n.

Group size. With g = 3, each counted group contributes 2 elements, so the kept side can be 2n/3 and the recurrence becomes T(n/3) + T(2n/3) + O(n). Since 1/3 + 2/3 = 1, that only gives O(n log n). With g = 7, each counted group contributes 4 elements, the kept side is at most 5n/7 and 1/7 + 5/7 = 6/7 < 1 keeps it linear.

Randomness. Median-of-medians is deterministic therefore on a given input every run makes exactly the same comparisons, so all of its timing variance comes from the machine. Quickselect's count varies from run to run on the same input, which is what the tail experiment measures.

2.4 Predictions

These are what I believed before running any timing. Some are instinct rather than calculation and I have said so where that is the case.

1. **Random input, time.** On instinct, median-of-medians will be about 2× slower than quickselect. My reasoning is that finding one given position does not require splitting evenly therefore a split biased towards one side can serve just as well and a random pivot is about twice as likely to land towards a side as near the centre.
2. **The library baseline.** std::nth_element is a quickselect variant, so I expect it to run at about the same speed as my quickselect.
3. **Crossover.** At very small n, quickselect has a higher chance of picking a pivot near an end, while median-of-medians should finish quickly. If median-of-medians wins anywhere on random input, I expect it to be at very small n.
4. **Sorted input.** Sorted input should make no difference to either algorithm, because both go through the same process whatever the order.
5. **Quickselect's tail.** The time distribution over 1000+ seeds will be slightly right-skewed: most runs a little faster than the mean, with a tail of slower runs. On instinct I would guess that the slowest 0.1% will take about 2 times the median time.
6. **Group size.** The three group sizes will differ in time and the differences will depend strongly on the size of n.
7. **Duplicates under Lomuto.** Quickselect should grow as n^2 on few-distinct input. In theory median-of-medians should stay linear, though I doubt that holds in practice.
8. **Cost of the fix.** Three-way's extra comparisons on random input will not show up in time by the same proportion: the time increase will be smaller, but not much smaller.

3. Experimental methodology

3.1 What I measured

I measured wall-clock time and key comparisons in separate builds so that counting could never slow down the timed code. Every key comparison in both algorithms goes through one function, less(), which increments a counter when the counted build defines SEL_COUNT_COMPARISONS and otherwise compiles to a plain a < b. The timing build therefore runs exactly the code a user would run while the counted build gives the comparison counts that Section 2's formulas predict. A test requiring at least n − 1 counted comparisons on every input catches any comparison that bypasses it.

3.2 Experiments and input families

The harness is a single C++ binary with the five modes in the table below, and the target rank is always the median, k = ⌊n/2⌋. Random input consists of uniformly random integers from 0 to 10n, few-distinct input draws every key from {0, 1, 2, 3}, all-equal input repeats one value and sorted input is already in ascending order. Sorted input tests whether order matters when quickselect's comparison count says it should not, and the duplicate-heavy families target the tie-handling weakness from Section 2.3.

Mode	Algorithms and schemes	Inputs and sizes	Purpose
sweep	Quickselect, median-of-medians (g = 5) and nth_element, three-way	Random, few-distinct and sorted, n = 10 to 10^7	Main comparison and crossover
groups	Median-of-medians with g = 3, 5 and 7, three-way	Random, n = 10^3 to 10^7	Effect of group size
fix	Both algorithms, Lomuto and three-way	Random, n = 10^3 to 10^7	Cost of the duplicate fix on random input
dups	Both algorithms, Lomuto and three-way	Few-distinct and all-equal, n = 250 doubling to 1,024,000	Behaviour under heavy duplicates
tail	Quickselect over 2,000 pivot seeds, plus 200 runs each of median-of-medians and nth_element	One fixed random input, n = 10^6	Run-to-run distribution of quickselect

3.3 Setup

All benchmarks ran on a laptop with an Intel Core Ultra 7 258V (four performance and four efficiency cores, 12 MiB of L3 cache) under WSL2 Ubuntu with kernel 6.6.87.2. I compiled with GCC 15.2.0 and CMake 4.2.3 as C++17 in Release mode with -O3 -DNDEBUG. Every benchmark ran on Linux even though I developed on Windows, because MSVC and libstdc++ implement std::uniform_int_distribution differently, so the same seed gives quickselect different pivots on each platform and mixing them would break reproducibility. The results come from commit 7eac51c, and scripts/reproduce.sh rebuilds everything, runs the tests, records the environment and regenerates every result and figure in about 15 minutes.

3.4 Fairness

Every input is generated from a seed built from a fixed base seed (20261103), the family, n and the repetition number, so every algorithm and both builds see exactly the same instances, and quickselect's pivot seeds are recorded so any run can be replayed. Generation, copying and result checking all sit outside the timed region, which is measured with std::chrono::steady_clock, and every result is checked against std::nth_element so a wrong answer can never produce a fast time. The algorithm order rotates each repetition, and one warm-up run on its own input is discarded before timing starts. Both algorithms share identical copies of the partition code, so the comparison measures pivot selection rather than unequal tuning effort.

Small inputs finish too quickly to time individually, so for small n each measurement times a batch of distinct inputs from a shared pool, sized to take about 5 ms and capped at about 200,000 keys. The inputs in a batch must be distinct because my first version batched identical copies, and the branch predictor learned the deterministic algorithms' patterns and made median-of-medians and nth_element look three to five times faster below $n = 10^4$ (Section 8). Those flawed results are kept in results_v1_batching_artifact/. To stop the quadratic Lomuto runs from taking hours, each series stops before the first size predicted to take more than 10 s and those sizes are logged as capped.

3.5 Repetitions and statistics

The timing build ran 15 repetitions for $n \le 10^5$, 9 for $n \le 10^6$ and 5 above that. The counted build ran 50 repetitions for the sweep, group and fix modes, the duplicates mode ran 3 and the tail ran once per seed. I report time as the median over repetitions with 25th to 75th percentile bars, because the median ignores the occasional run slowed by the operating system. I report comparisons as the mean with standard-error bars, because Section 2's formulas are expected values and quickselect's count varies widely from run to run.

The main limitation is that Windows schedules WSL's virtual CPUs onto physical cores itself, so I could not pin runs to a performance core and a run that lands on an efficiency core or gets interrupted shows up as noise. Taking medians limits this, and the deterministic baselines in the tail experiment show how much of the spread comes from the machine rather than from quickselect's randomness.

4. Results and analysis

Times are medians from the timing build and comparisons are means from the counted build (Section 3), and input is random unless stated otherwise.

4.1 Random input

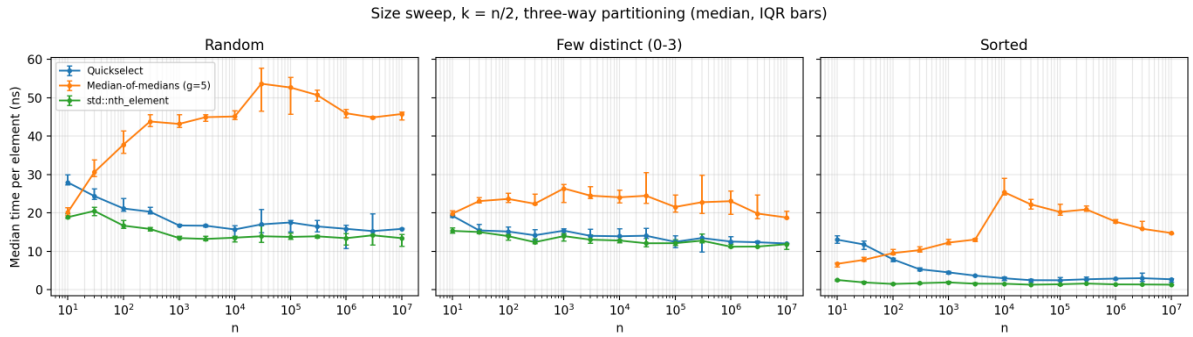


Figure 1. Median time per element against n, by algorithm and input family.

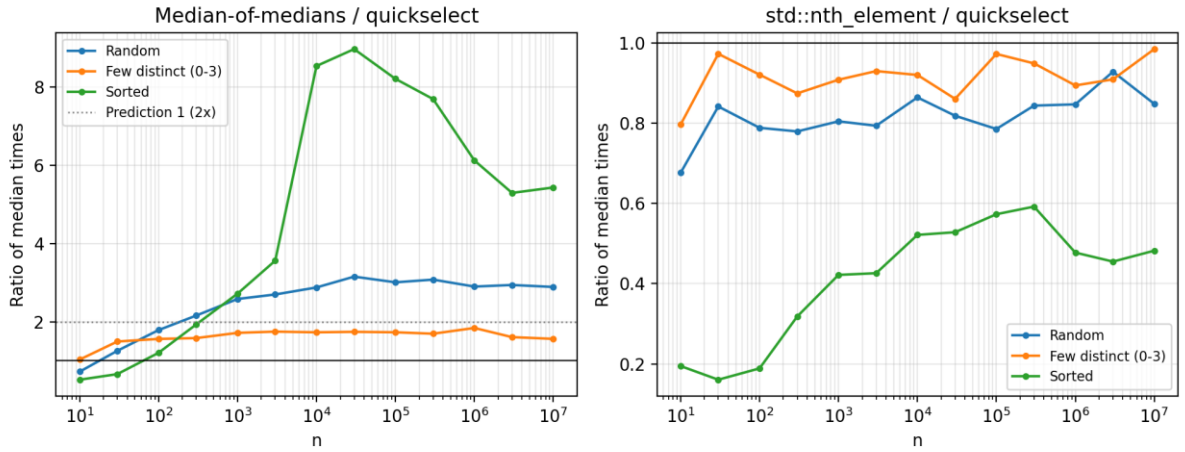


Figure 2. Time relative to quickselect on random input, with the predicted 2× ratio marked.

Median-of-medians was 2.58 times slower than quickselect at n = 10^3, 3.01 times at 10^5 and 2.89 times at 10^7, so the predicted 2× gap was too optimistic at every size from 10^3 up. Table 4.1 shows why. Median-of-medians made about 10.3 comparisons per element, close to the 10.1 from Section 2.2's model, and quickselect made 5.0 to 5.3 against an expected 5.1, so the comparison ratio is the 2 the prediction assumed and the extra time comes from work that comparisons do not count. Every round of median-of-medians reads the whole window for the group sorts, writes the n/5 medians to the front and recurses on them before it partitions, while quickselect partitions straight away. Its insertion sorts also branch unpredictably on random data, and at n = 10^7 the 80 MB array is far larger than the cache, so every extra pass is paid for in memory traffic.

Table 4.1. Comparisons and time per element at n = 10^7.

Algorithm	Comparisons per element, random	Comparisons per element, sorted	ns per element, random	ns per element, sorted
Quickselect	5.33	5.20	15.84	2.72
Median-of-medians (g = 5)	10.28	8.96	45.76	14.77
nth_element	2.72	2.50	13.43	1.31

4.2 The library baseline and small inputs

nth_element was faster than my quickselect at every size, taking 0.68 times its time at n = 10, 0.80 at 10^3 and 0.85 at 10^7, rather than running about even as predicted. Its Hoare-style partition compares each key with the pivot about once where three-way partitioning often needs two, and its median-of-three pivot tends to split closer to the middle, so it makes about 2.7 comparisons per element against 5.3. Halving the comparisons only saved 15% at 10^7, which again points to memory passes as the main cost.

Median-of-medians was faster than quickselect at n = 10, taking 0.72 times its time on random input and 0.52 on sorted input, which matches prediction 3. Quickselect took over from n = 30 on random input, while on sorted input median-of-medians was still ahead at n = 30 (0.66). At these sizes the fixed cost of each round outweighs the work per element, and quickselect draws a random number for every pivot while median-of-medians on ten keys does little more than two insertion sorts and a short partition.

4.3 Sorted input

Prediction 4, that sorted input would make no difference, held for comparisons but not for time. Quickselect's count barely moved because a random pivot's rank is uniform whatever the order, and median-of-medians fell from 10.28 to 8.96 because a sorted group of five costs 4 comparisons instead of about 7.7 (Section 2.3). Time fell much further, by a factor of about 5.8 for quickselect, 3.1 for median-of-medians and 10.3 for nth_element. The most likely cause is branch prediction and data movement. On sorted input every key below the pivot already sits to its left, so the partition’s comparison goes one way for a long run and then switches once, whereas on random input the same branch is close to a coin flip. nth_element gains the most because on sorted input its median-of-three pivot is the window's middle key, so its partitions split evenly as well as predictably. Median-of-medians gains the least because its group sorts, median gathering and recursion make the same extra passes over memory whether or not the data is sorted. I did not measure branch mispredictions directly, so this explanation is inferred rather than confirmed with hardware counters.

Few-distinct input was also faster than random input for all three algorithms at n = 10^7, at 12.04 ns per element for quickselect, 18.80 for median-of-medians and 11.85 for nth_element. With only four

distinct values the three-way partition's equal block is large and k usually lands in it within a round or two, which cut quickselect's comparisons from 5.33 to 2.88 per element.

4.4 Quickselect's tail

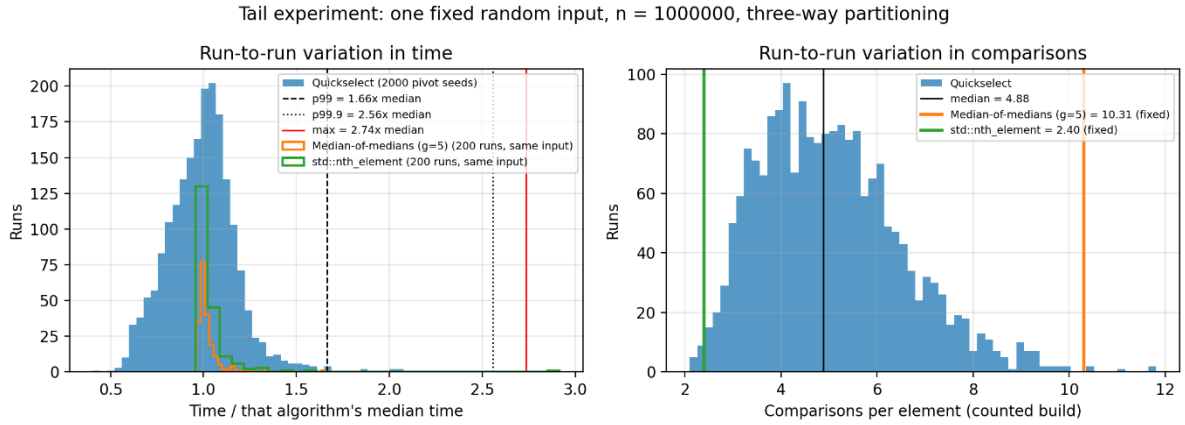


Figure 3. Quickselect's time and comparisons over 2,000 pivot seeds on one input of 10^6 keys.

Quickselect's time was right-skewed as predicted, with a skewness of 1.81, p99 at 1.66 times the median, p99.9 at 2.56 times (resting on about two runs) and a maximum of 2.74 times. The deterministic baselines show how much of this comes from the machine. Median-of-medians and nth_element make exactly the same comparisons on every run, yet their slowest runs took 1.66 and 2.92 times their medians, with skewness of 5.2 and 9.2. Their noise takes the form of a few isolated spikes, while quickselect's comparison count, which machine noise cannot affect, has a broad and moderate skew (0.70) and ranges from 2.1 to 11.8 comparisons per element around a mean of about 5.0. Quickselect's time tail is therefore a mix of genuine randomness and ordinary scheduling noise. Even its worst run did only about 2.4 times the median amount of work, and its slowest run, at 43.1 ms, was still faster than median-of-medians' fastest run on the same input, at 43.2 ms. This fits the argument in Section 2.1 that long runs of bad pivots become exponentially unlikely.

4.5 Group size

Table 4.2. Median-of-medians by group size at $n = 10^7$.

Group size	ns per element	Comparisons per element
g = 3	63.5	14.12
g = 5	42.6	10.28
g = 7	40.4	10.38

With g = 3 median-of-medians was about 1.5 times slower than with g = 5, and its comparisons per element were still rising with n, consistent with the O(n log n) bound from Section 2.3 because 1/3 + 2/3 = 1 leaves the recurrence no room to shrink. g = 7 was slightly faster than g = 5 despite making slightly more comparisons. Its larger groups cost more comparisons inside the insertion sorts, but it recurses on n/7 medians instead of n/5 and so gathers and revisits less data.

4.6 Duplicates and the cost of the fix

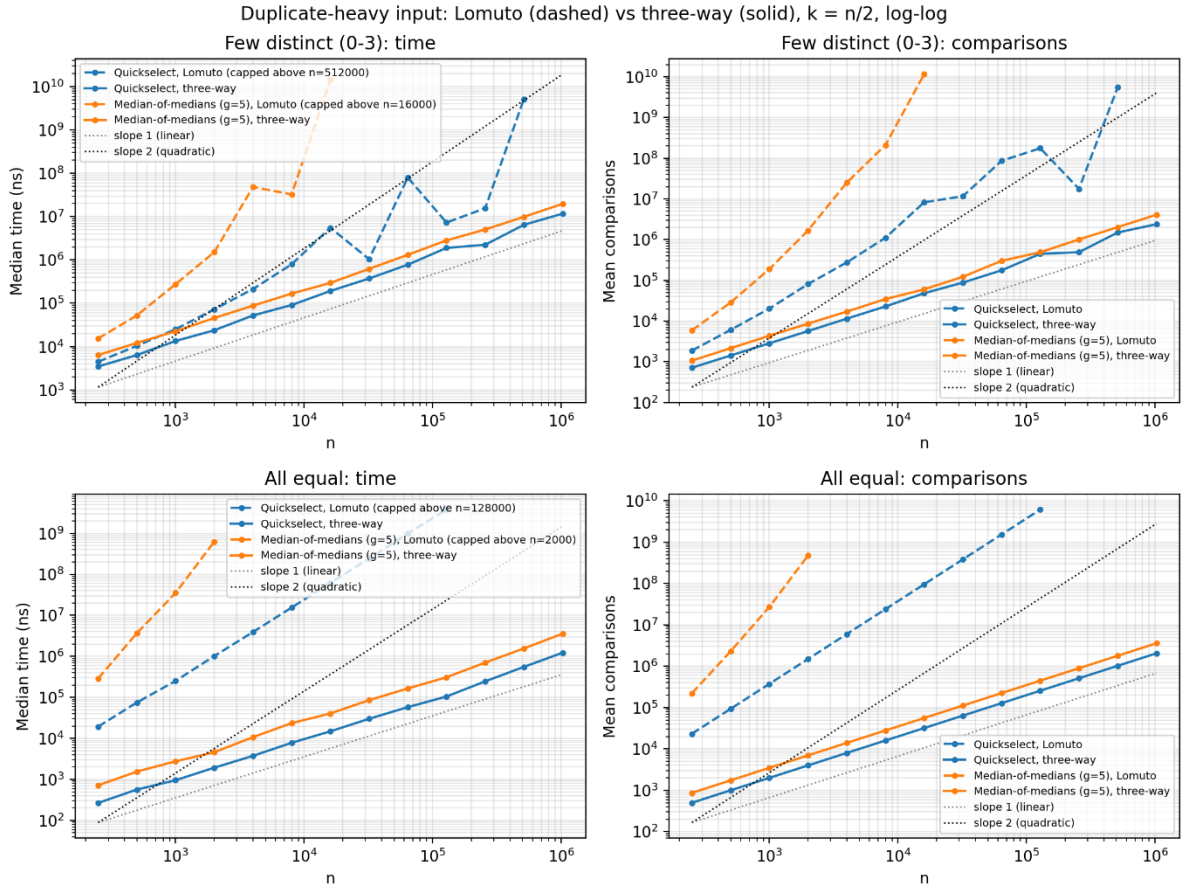


Figure 4. Time and comparisons on duplicate-heavy input under both partition schemes (log-log).

On all-equal input the comparison count of Lomuto quickselect had a local log-log slope of 2.00 at every size, the cleanest quadratic in the project. Lomuto sends every key equal to the pivot left, so each round removes only the pivot and reaching the median takes about $3n^2/8$ comparisons, and randomness cannot help because every pivot has the same value. Lomuto median-of-medians was worse still, with a comparison slope rising from 3.37 to 3.56 to 4.13 before the 10 s cap stopped the series above n = 2,000, which is the rising slope that the $m^{\Theta(\log m)}$ derivation in Section 2.3 predicts. Under three-way partitioning both algorithms became linear, at exactly 2n comparisons for quickselect and 3.5n for median-of-medians at large n. The few-distinct Lomuto curves are jagged because k = n/2 lands near the boundary between the run of 1s and the run of 2s in sorted order. Section 2.3 explains why median-of-medians fails so badly here.

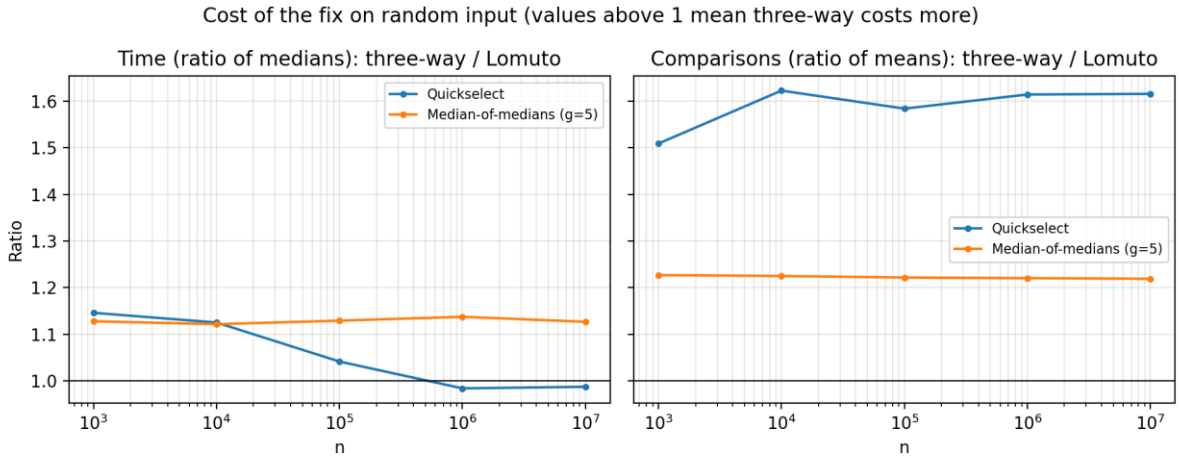


Figure 5. Three-way partitioning relative to Lomuto on random input.

On random input the fix is nearly free. Three-way partitioning added 51% more comparisons to quickselect at n = 10^3 and 62% at 10^7, but its time rose by only 15% at 10^3 and fell by 1% at 10^7. For median-of-medians comparisons rose by 22 to 23% and time by 13%. The second comparison works on a value already loaded from memory, so once memory traffic dominates it costs almost nothing, which confirms prediction 8 more strongly than expected.

4.7 What ties the results together

The comparison counts matched Section 2's formulas closely but were a poor guide to time. The algorithms differ by 2 times in comparisons but nearly 3 in time, sorted input cuts time up to tenfold with almost the same comparisons and three-way partitioning adds 62% more comparisons for no extra time. In each case time follows the number of passes over memory and how predictable the branches are, which the comparison model leaves out.

5. What surprised me

5.1 Sorted input

The result that surprised me most was sorted input. I predicted it would make no difference (prediction 4), and my reasoning seemed simple. Neither algorithm knows the array is sorted, so it makes the same comparisons either way, and I assumed the same comparisons would take the same time. The comparison counts proved the first half of that right, since quickselect's barely changed, but at n = 10^7 its time fell from 15.84 to 2.72 ns per element and nth_element's fell almost tenfold (Table 4.1). When I first saw this I was confused, because the measurement I trusted most said nothing had changed while the clock said almost everything had.

My best explanation after a little bit of research is that a comparison has no fixed cost, because its cost depends on whether the processor can guess the result. A modern CPU works on many instructions at once, and when it reaches a branch such as "is this key less than the pivot?" it guesses the answer and keeps going rather than waiting. A correct guess makes the comparison almost free, while a wrong guess forces the CPU to throw away the work it started and costs something like 10 to 20 cycles. On random input with a pivot near the median each answer is close to a coin flip, so the predictor is wrong about half the time and the partition loop pays that penalty again and again. On sorted input the answers come as a long run of "yes" followed by a long run of "no", so the predictor is almost always right. The comparisons are identical but most of their hidden cost disappears.

This also fits the differences between the algorithms. nth_element gains the most because on sorted input its median-of-three pivot is the middle key, so its partitions split evenly as well as predictably. Median-of-medians gains the least and I am less sure why, but my guess is that more of its time goes into fixed overhead that sorted input cannot remove, such as setting up thousands of tiny insertion sorts and moving the medians to the front of the window. I did not measure branch mispredictions directly, so this explanation fits the data rather than being confirmed by it, and Section 6 describes how I would test it.

5.2 Three-way partitioning

The second surprise is the mirror image of the first. Three-way partitioning made quickselect do 62% more comparisons at n = 10^7, and I expected that to cost a matching amount of time, but its time fell by 1%. I think the extra comparisons are the cheap kind. Both schemes first ask whether a key is below the pivot, which is the unpredictable coin flip, and three-way partitioning then asks whether the key equals the pivot. On random keys that answer is almost always "no", so the predictor gets it right nearly every time. At n = 10^3 the whole array fits in cache and the CPU is kept busy executing instructions, so the extra work shows up as a 15% slowdown. At n = 10^7 the 80 MB array is far larger than the 12 MiB L3 cache, so the CPU spends much of its time waiting for data from memory and the extra comparisons fit into time it would otherwise spend idle.

5.3 What this changed for me

Both surprises came from the same assumption, that more comparisons means more time and the same comparisons means the same time, which just seemed logical to me because more operations should take longer. That assumption runs through Section 2, where every prediction started from a comparison count, and it explains why my predictions about comparisons were accurate while my predictions about time were not. I still think comparison counts are the right way to analyse these

algorithms, since they matched the theory to within a few percent, but they describe the work an algorithm asks for rather than what that work costs on real hardware. Whether a branch is predictable and whether the data is already in cache matter as much as how many comparisons are made, and both depend on the input as well as on the algorithm.

6. What I'd do next

The first thing I would do is test the explanation in Section 5 directly. Running the benchmarks under perf stat would count branch mispredictions and cache misses, and if the explanation is right, quickselect's mispredictions per element should collapse on sorted input while its comparisons stay the same. Hardware counters may not be available inside WSL2, so this would probably need native Linux, which would also let me pin runs to a performance core. A second test would be a branchless partition that uses conditional moves instead of branches, because if branch prediction explains the gap, a branchless version should run at nearly the same speed on random and sorted input.

I would also like to repeat the comparison with keys that are expensive to compare, such as long strings. My results suggest comparison counts are a poor guide to time when comparisons are cheap 64-bit integers, but with expensive comparisons the 2× comparison ratio between the algorithms might finally become a 2× time ratio. Finally, I would investigate the jump in median-of-medians' time on sorted input between n = 3,000 and 10^4 (Figure 1), which none of my explanations account for. All of these extensions would cover all the additional questions that got opened by my results and make it more evident as to the difference between theoretical and practical which is a large part of this comparison.

7. Individual contributions (completed the assignment solo, so skipped as per Troy's instructions)

8. AI use

8.1 How I used AI

I used Claude throughout the project in three roles. It was first a tutor, introducing ideas such as partition schemes and branch prediction well enough that I could then research them myself. It was also an infrastructure writer that produced the project skeleton, the tests, the benchmark harness, reproduce.sh and plot.py, all of which I reviewed and ran. Finally, it was a drafter that helped plan the report and drafted most sections, the video script and the slides, which I then edited and finalised. I typed all of the algorithm code myself and every prediction in Section 2.4 is my own. The full record is in ai-log.md, and the algorithm code I wrote on 27 and 28 October was committed later, on 2 November.

8.2 Where it was wrong

These are the three most significant of the five problems in my log.

The first was the Lomuto-based median-of-medians adaptation Claude gave me, which it reported as passing every test. The answers were correct, but my Debug test run had not finished after 15 minutes. My intuition told me something was wrong and so I questioned it, and Claude found that Lomuto sends every key equal to the pivot to one side, which breaks the 3n/10 guarantee on duplicates (Section 2.3), so I switched both algorithms to three-way partitioning.

The second was in the benchmark harness, which timed small inputs in batches of identical copies. The branch predictor learned the deterministic algorithms' branch patterns and made median-of-medians and nth_element look three to five times faster below $n = 10^4$. The results looked convincing until Claude noticed a jump at the batching boundary in the first version of Figure 1 and confirmed it with a micro-benchmark. The harness now batches distinct inputs, and the flawed run is kept in results_v1_batching_artifact/.

The third was Claude's draft of Section 4, which took several numbers from its own notes rather than the data, such as calling comparison slopes time slopes. I found the mismatches by checking the draft against summary.csv and corrected them before merging it.

More examples and further info are found in the repo's ai-log.md.

8.3 AI in place of a collaborator

As a stand-in for a group member, Claude was most useful for bouncing ideas around, cutting the workload and reviewing code. It challenged predictions that were too vague, explained ideas I could not explain myself and caught two of its own harness bugs.

Where it fell short was backbone. A teammate would argue against a bad decision, but Claude often went along with mine even when, looking back, they were clearly wrong. The clearest case was Lomuto. I chose it because it was the version in my research, which I assumed was standard, and when I asked whether it would work for median-of-medians Claude answered with a confident yes which put me in a position of overconfidence instead of how a teammate would've maybe questioned it and forced me to figure it out myself or with their help. In the end its wrong answers sounded just as certain as its correct ones, so its confidence told me nothing. The lesson I took from this is that with AI as a collaborator the reviewing falls entirely on me and I need to be less trusting.

References

Blum, M., Floyd, R. W., Pratt, V. R., Rivest, R. L., & Tarjan, R. E. (1973). Time bounds for selection. *Journal of Computer and System Sciences, 7*(4), 448–461. https://doi.org/10.1016/S0022-0000(73)80033-9

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Edge. (2012, June 2). *Quickselect algorithm simplified explanation* [Online forum post]. Stack Overflow. https://stackoverflow.com/questions/10861388/quickselect-algorithm-simplified-explanation

GeeksforGeeks. (2024, August 14). *Quickselect algorithm*. https://www.geeksforgeeks.org/dsa/quickselect-algorithm/

GeeksforGeeks. (2025, July 23). *K'th smallest/largest element in unsorted array: Worst case linear time*. https://www.geeksforgeeks.org/dsa/kth-smallest-largest-element-in-unsorted-array-worst-case-linear-time/

Hoare, C. A. R. (1961). Algorithm 65: Find. *Communications of the ACM, 4*(7), 321–322. https://doi.org/10.1145/366622.366647

Knuth, D. E. (1972). Mathematical analysis of algorithms. In *Information processing 71: Proceedings of IFIP Congress 1971* (pp. 19–27). North-Holland.

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience, 27*(8), 983–993. https://doi.org/10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#

Sayle, R., & Carlini, P. (2006, August 28). *[Patch] Change std::nth_element to O(N log N) worst case* [Online forum post]. libstdc++ mailing list archive. https://gcc.gnu.org/ml/libstdc++/2006-08/msg00028.html

simon. (2012, February 28). *Understanding "median of medians" algorithm* [Online forum post]. Stack Overflow. https://stackoverflow.com/questions/9489061/understanding-median-of-medians-algorithm